

Generative Adversarial Networks for photo to Hayao Miyazaki style cartoons

Piyush Sawarkar
IIT, Bombay
22m0772@iitb.ac.in

Kiran kumar kunchu
IIT, Bombay
22m0755@iitb.ac.in

Marotu Pavan Kumar
IIT, Bombay
22m0767@iitb.ac.in

Sanyasi Sai Sandeep Pothala
IIT, Bombay
22m0792@iitb.ac.in

Ayush Kumar Singh
IIT, Bombay
22m0793@iitb.ac.in

Abstract—The paper implemented in this project takes on the problem of transferring the style of cartoon images to real-life photographic images by implementing previous work done by CartoonGAN. We trained a Generative Adversarial Network (GAN) on Safebooru dataset available on kaggle, being closest alternative for Hayao Miyazaki style artwork produced at Studio Ghibli, used in the paper.

Index Terms—cartoon, anime, Generative Adversarial Networks, style transfer

I. INTRODUCTION

Cartoons are creative forms with a wide range of both style and topic. Cartoon development often takes a lot of work and artistic talent. Automating the transformation of real-world photographs into cartoon-style drawn graphics can be done using the machine learning idea of style transferring neural networks.

Pairings and unpairings are two ways to partition the style transmitting neural network field. A source image (in this case, a photograph of real-life activity) and a target style image (a cartoon, in this case) are both shown in the paired approach. This method calls for a particularly particular kind of pre- and post-styled photos. Because of this, obtaining datasets for this strategy may be challenging. The unpaired technique, in comparison, uses two distinct sets of images: one set of source images (from real world) and one set of target images (from cartoons). A model is trained to create images from the two image sets while maintaining the source content’s organisational structure and implementing the goal style.

Generative Adversarial Networks (GAN) are a well-liked technique for unpaired style transfer. The fundamental principle behind GANs is that by exploiting previously seen data, they enable us to create samples from novel, unexplored data distributions. With enough information and processing power, it has been discovered that it is possible to replicate visual features that are eerily similar to the target. When there is a lack of data in the domain, many see the potential of using GANs to generate new data. - To automate the process of producing cartoon graphics, Y. Chen, Y. Lai, and Y. Liu put out the CartoonGAN approach. By using a convolutional neural network to analyse real-world images and applying the visual

language of current cartoons to them. Their methodology is centred on combining convolutional neural networks (CNN) with GANs. Their research is based on a strategy that was previously put forth by L. A. Gatys, A. S. Ecker, and M. Bethge .

S. Hicsonmez, N. Samet, E. Akbas, and P. Duygulu put out the GANILLA approach, which uses GANs and is educated on illustrations from children’s books as well as the aesthetic of Hayao Miyazaki. They also suggest a quantitative evaluation technique that divides style and content transfer into two CNN categories. With a larger dataset, we want to construct our own approach based on CartoonGAN and compare it to CartoonGAN and GANILLA utilising each network’s pretrained weights for the Hayao Miyazaki style.

II. IMPLEMENTATION

For real-world photos, we chose to use the Flickr30K dataset from Flickr.com¹, which consists of 31 783 crowd-sourced images. The bounding boxes of the photographs are annotated with descriptions of what they individually depict. We only use the raw photos in this work and discard the comments.

A. Datasets

For real-world photos, we selected the Flickr30K. dataset, a crowdsourced collection of 31 783 photos, Flickr.com. The pictures have bounding boxes with annotations. that each image stands for. For this project, we ignore the and only use the unaltered photographs.

In the original paper, the data was created by removing images from anime films, such as Miyazaki Hayao’s ”Spirited Away.” We searched for an alternate dataset due to copyright concerns and found it in the safebooru dataset from Kaggle. The numerous picture makers and their various artistic approaches provide a problem for this process. This will produce training data that is challenging to learn from. We believe that this won’t adequately train the network, but We’ll give it a shot. The paper solely employs one particular artist’s style.

The model is trained using an edge-smoothed version on each cartoon image in order to help the GAN learn to produce sharp edges in the cartoon image. The edges in the paper are

first located using a technique called canny-edge, after which they are dilated and smoothed using gaussian smoothing. In my version, We use OpenCV to perform the canny edges, dilation, and gaussian blur. We also make the white background transparent and use Pillow to paste the edges back onto the original image.

B. Network Architecture

PyTorch was used to implement the model . A generator G and a discriminator D make up a GAN in general. To be able to identify which photos are authentic cartoons, the discriminator is trained on both the style images and the outcomes of the generator. In order to produce a result that can deceive the discriminator, the generator must discover a style to apply to the source image.

The cycle is then repeated after the discriminator has been trained using the generator's results. The networks are always competing with one another and getting better over time.

The generator network, which converts real-world images into cartoon equivalents, has 14 layers or blocks: 1 convolution layer, 2 blocks for down-convolution, 8 blocks for residual convolution, and 2 blocks for up-convolution. The discriminator network has five layers: a flat convolution layer, two down-convolutions, a feature creation block, and a convolution layer. It can identify a picture as either realistic or cartoon.

In order to create a set of classification results, one for each patch, the discriminator is what is known as a patchGAN, which means that rather than providing a single classification for each image, it is used on smaller, cropped areas (patches) of each image. Leaky ReLU (LReLU) is used here with $\alpha = 0.2$ after every normalisation.

C. Loss Functions

A loss function is used to evaluate the error, and therefore the performance of a model. In the context of GANs it is used to evaluate the performance of both networks . Y. Chen, Y. Lai, and Y. Liu proposes the use of a combined loss function $\mathcal{L}(G, D)$ which takes into account both adversarial loss $\mathcal{L}_{adv}(G, D)$ and content loss $\mathcal{L}_{con}(G, D)$ for both networks G and D .

$$\mathcal{L}(G, D) = \mathcal{L}_{adv}(G, D) + \omega \mathcal{L}_{con}(G, D)$$

ω is a scalar used to balance the two loss functions. , we set $\omega = 10$.

The content loss can be considered as a measure of how effectively the source image's content structure is preserved, whereas the adversarial loss can be seen as a measure of how well the combined networks can transfer the cartoon style to the target image. Optimizing the adversarial loss is a *minimax problem* where the discriminator network tries minimize the chance to do a wrong prediction, whereas the generator network aims to maximize the probability of fakes that the discriminator misclassifies as real.

1) *Adversarial Loss:* The adversarial loss $\mathcal{L}_{adv}(G, D)$ This turns on the generator and transforms the photo of the image into a comic book style. The result's value indicates whether or not it resembles a cartoon image. The study emphasises that in order to achieve clear edges in the finished result, crisp edges, which are a distinguishing element of cartoon images and are a small aspect of the image, must be kept. The discriminator in the paper is trained with cartoon images that have smoothed edges in addition to teaching it to be able to distinguish between clear and smooth edges, which solves this issue. The authors state that this is done using the edge-promoting adversarial loss function:

$$\mathcal{L}_{adv}(G, D) = \mathbb{E}_{c_i \sim S_{data}(c)}[\log D(c_i)] + \mathbb{E}_{e_j \sim S_{data}(e)}[\log(1D(e_j))] + \mathbb{E}_{p_k \sim S_{data}(p)}[\log(1D(G(p_k)))]$$

This is the loss function formula for the discriminator because its output has no bearing on the content loss component of the loss function.

This portion of the formula is not utilised during the initialization stage of the generator, as stated in the study.

Only the portion of the generator's loss function that the generator can influence is used for the training phase of the generator:

$$\mathbb{E}_{p_k \sim S_{data}(p)}[\log(1D(G(p_k)))]$$

2) *Content Loss:* The content loss $\mathcal{L}_{con}(G, D)$, which preserves the semantic content during transformation. To calculate this, in the paper the high-level feature maps of the VGG network is used, in particular the layer $l = \text{conv4_4}$. The output of the layer l for the original photo is subtracted from the output of the layer l of the generated image. The result is regularized using the $\mathcal{L}_{\infty}$ sparse regularization ($\|\dots\|_1$):

$$\mathcal{L}_{con}(G, D) = \mathbb{E}_{p_i \sim S_{data}(p)}[\|VGG_l(G(p_i)) - VGG_l(p_i)\|_1]$$

This part of the formula plays a role in the loss function for the generator, not for the discriminator, because only the generator is used within this formula.

3) *Loss function implementation:* In our initial implementation, We used all three of D's outputs—D(cartoon image), D(smoothed cartoon image), and D(G(photo))—as inputs to the discriminator loss. This gave me a discriminator output of batch size batch size x 1 x 64 x 64. The adversarial loss only returns a single value because it outputs a probability that indicates whether the input is fraudulent or not. We applied the loss function to the input tensor, which had the following shape: batch size x 1 x 64 x 64.

$$\mathcal{L}_{adv}(G, D) = \mathbb{E}_{c_i \sim S_{data}(c)}[\log D(c_i)] + \mathbb{E}_{e_j \sim S_{data}(e)}[\log(1D(e_j))] + \mathbb{E}_{p_k \sim S_{data}(p)}[\log(1D(G(p_k)))]$$

Calling $\log()$ directly with this value results in an error since the discriminator output can occasionally contain negative numbers. Consequently, I encapsulated the input of $\log$ in

abs(). We returned to the loss functions because my training results weren't what We had anticipated. My discriminator produces a tensor with the shape batch size $\times 1 \times 64 \times 64$, but an adversarial loss produces a probability and, consequently, a single value.

Therefore, either an activation function is required or We misimplemented the discriminator and it should output a probability, in order to obtain a probability from this tensor. We gave this some thought before using an activation function. To do this, We may either use a loss function like BCEWithLogitsLoss, which combines an activation function with loss, or We can activate the output of the discriminator's last layer. But which activation method should We employ?

Using sigmoid or softmax is an excellent option because the discriminator should provide a probability and only has two classes as outputs, real or fake. Both binary classification and classification of n-classes can be done using Softmax.

We first chose to employ a loss function, which mixes activation and loss function, and this provided me with the option of:

BCEWithLogitsLoss: Sigmoid and binary cross entropy loss
 CrossEntropyLoss: Softmax and negative log likelihood loss
 Which loss is best for a minimax problem's solution?

"This would necessitate revisions to model weights utilising stochastic ascent rather than stochastic descent if [minimax] were implemented directly. It is often implemented as a classic binary classification problem with the labels 0 and 1, respectively, for produced and real images."

see [12] Therefore We chose BCEWithLogitsLoss.

We used BCEWithLogitsLoss three times, once for each individual input, and added the values because it has two parameters: one for the input and one for the target.

However, when using this technique, the generator generates values that are less than zero. When attempting to convert these values to RGB, this created issues. Therefore, We choose to use sigmoid directly in the generator and discriminator as well as BCELoss as the loss function rather than combining activation and loss function.

III. EXPERIMENT

Each image was cropped to 224x224 pixels in size. We trained it for 10 epochs after replicating the initialization step carried out by Y. Chen, Y. Lai, and Y. Liu. A batch of 11 photos were used to train each cycle. AdamW, an optimizer that is an enhancement on D. P. Kingma and J. L. Ba work was employed. The learning rate was set to a constant 1×10^{-3} and a weight decay of 1×10^{-4} . We also used a cyclic learning rate scheduler with the maximum learning rate set to 1×10^{-2} . Both networks were trained for a total of 60 epochs using the entire flickr30k dataset.

A. Training round 1

Initially, We set ω , It adds a weight to the paper's stated value of 10 in order to balance the paper's aesthetic and content preservation. Although the created graphics lack cartoon-like characteristics, the content preservation was quite good

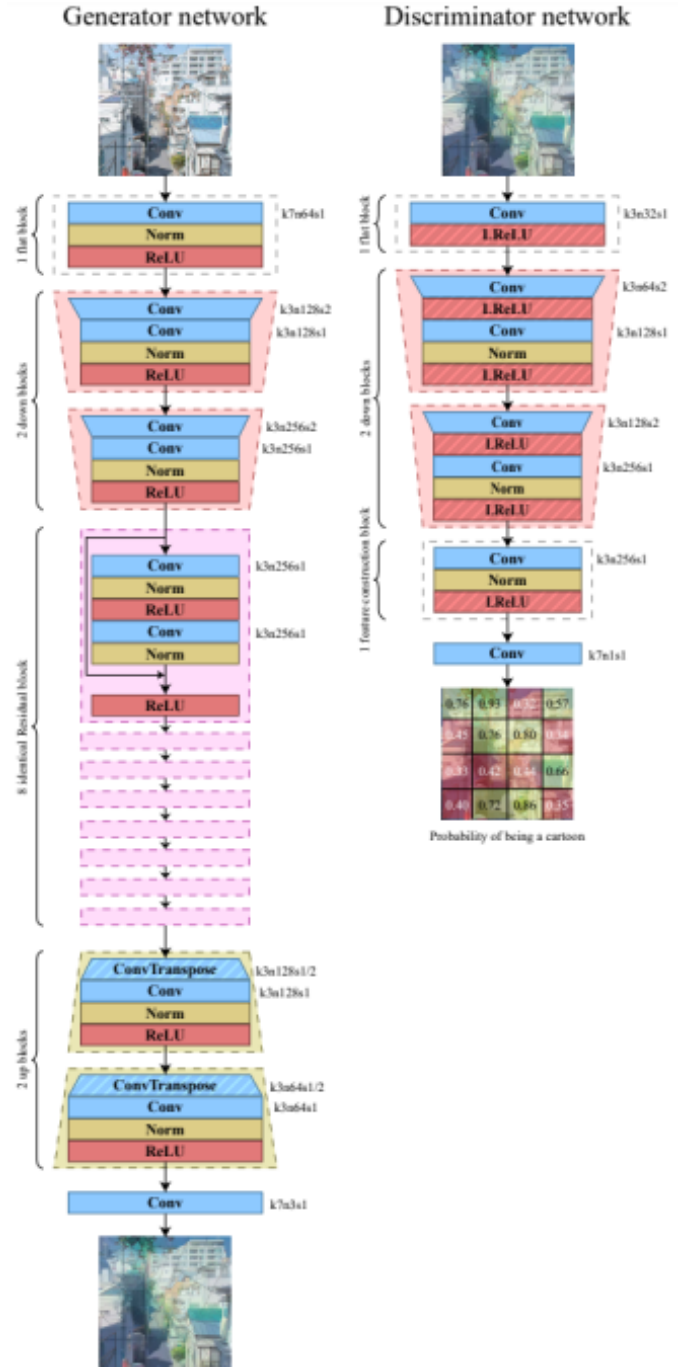


Fig. 1. GAN Architecture

after 210 iterations. This might be a problem with my input data, as We used various cartoon designs from various artists rather of just one, as was done in the study. Because of this, the content loss is far greater than the adversarial loss. Perhaps We underestimated the content loss? Has it have to be much lower? We concentrate on the adversarial loss since the generated images effectively preserve the content. Maybe We'll be using the incorrect VGG-Model. Since there is no issue with image preservation, my focus is on comic

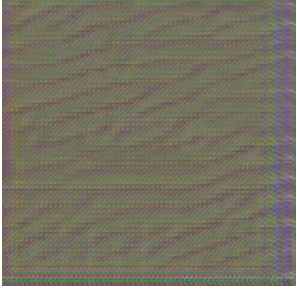


Fig. 2. direct after start, one of the first epochs



Fig. 3. direct at the beginning of epoch 11 with use of full generator loss instead of init loss. These results seem to be outliers at this stage of training due to the next outputs look more similar like the inputs

style. As the comedic impact results from the adversarial loss, we attempted a much lower ω , to balance the values of $g_content_loss$ and $g_adversarial_loss$ on an equal level for the next training round.

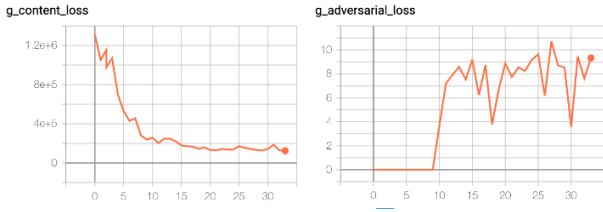


Fig. 4. Examine the parts of the generator loss over time, the following values are observable

B. Training round 2

As $g_content_loss$ has values of $4e+5$, We choose $\omega=0.00001$. After 210 epochs, the result is the following:

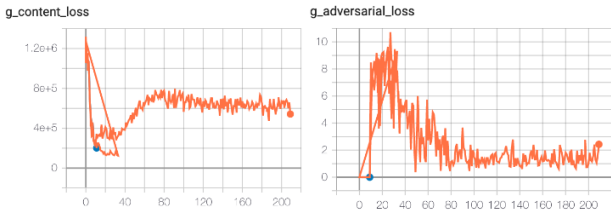


Fig. 5. Examine the parts of the generator loss over time, the following values are observable

In compared to training round 1, where was substantially greater and hence played a larger influence in the overall loss, the optimization of the content loss lags a little behind. On the other hand, because it contributes more to the overall loss, the adversarial loss optimization improves. The comic effect begins to take effect as can be seen in the sample images, but the text is still kept.

We set $\omega=0$ for the following trial so that the optimization would only have an impact on the adversarial loss.

C. Training round 3

With $\omega=0$, the trained model loses the content information but the comic-style hits through: not part of the loss result due to set to zero, but plotted here to visualize!

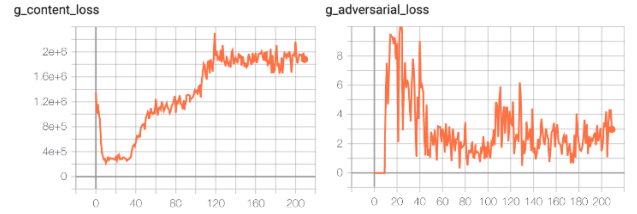


Fig. 6. As comic-style starts to hit through around $\omega=0.00001$ and has full dominance at $\omega=0$, I try some between these values for the next round.

D. Training round 4

Set $\omega=0.00001/2=0.000005$ Unfortunately, before retraining, We neglected to clean up my tensorboard results folder, so the results are plotted against those from round 3. Through manual image processing, We attempted to make it a little bit easier to view. The actual plot from round four is in orange; the green plot is the outdated one from round three.

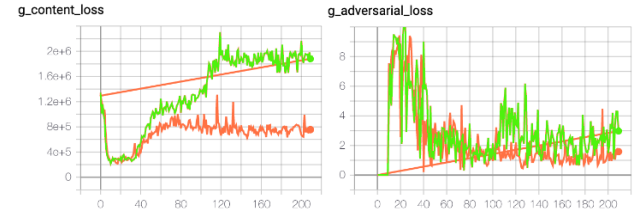


Fig. 7. Examine the parts of the generator loss over time, the following values are observable

IV. EVALUATION

Evaluation of GANs can be challenging, particularly when applied to style transfer where the outcome is an altered image. Two neural networks are employed in the proposed quantitative evaluation to assess, respectively, how well the style of the target image is transferred and how well the content of the source image is preserved (Content-CNN). They also suggest a technique for qualitative assessment, selecting the model with the best "style and content identifiability" by contrasting 4 different style transfer GANs. We compare CartoonGAN, GANILLA, and our approach, three different implemented

models^{4,5} of cartoon style transfer GANs. We employ weights tuned to Hayao Miyazaki’s artwork for all of our models. Our team created a survey to compare the three models. We presented the subject with three images for each task, one from each individual model and one from the shared source image. On the basis of a given statement, the participant was then required to rank the three photos on a scale of 1-3. Here, We agrees with the offered statement the most, while 3 disagrees the least. For the inquiries, we requested that the participant to ”rank the following images according to aesthetic/visual appeal”(henceforth referred to as Aestheticness) and to ”rank the following images according to how much the image resembles a cartoon (i.e. illustrated photo or anime)”(henceforth referred to as Cartooniness). We created a straightforward web page hosted on GitHub Pages⁶ to conduct the poll, and we used Pageclip⁷ to compile the findings. For the survey, we used a total of 20 source images—10 for each question and one for each task. For each participant, the order of the images (and corresponding activities) was randomly assigned to both questions. Every source image thusly receives responses to both inquiries. Fig. 2 shows examples of the survey views. The 20 source photographs came from Unsplash.com, where they were downloaded. The code repository page contains these photos as well as the outcomes of CartoonGAN, GANILLA, and our approach.

V. FUTURE WORK

Pictures with a lot of foreground material could Reduced proportions of facial characteristics are used in general. It can also be fascinating to entirely remove the front text and merely transmit the background aesthetic. such as landscapes. The cartoon dataset’s smoothing of the edges utilised for preprocessing may be improved to be more universal. separate thresholds for each image, or even thresholds. These recommended improvements could also be accomplished through training. a model, particularly when working with a dataset as large as in In our situation, manual refining is not practical. Additional research could be done utilising various for studios or artists to assess the reproducibility of Y. Chen, Y. Lai, and Y. Liu. The company CoMix Wave Films, which has created a sizable repertoire of well-known films and is used as training material in [3], is home to the intriguing artist Makoto Shinkai. It would also be interesting to investigate additional architectures using the Y. Chen, Y. Lai, and Y. Liu methodology. The U-Net network architecture is an intriguing one since it enables the usage of lengthy shortcut connections. builds their model using the U-Net architecture. A huge dataset and the U-Net architecture combined with our methodology and CartoonGAN’s approach could result in more reliable findings. The network could also be trained over a longer period of time. There were time and computational resource restrictions on this experiment. The one graphics card described in subsection IV-A was the only resource we possessed.

VI. CONCLUSION

The network might also undergo longer-term training. This experiment was subject to time and computational resource constraints. The only resource we had was the single graphics card mentioned in part IV-A. The results of our model revealed intriguing characteristics that weren’t apparent in the results of the comparable models. The components include uniformly coloured regions and recognisable facial features from cartoons. Developing the cartoon dataset and experimenting with more durable architectures are other, interesting advancements.

REFERENCES

- [1] Filip Andersson, Simon Arvidsson <https://arxiv.org/pdf/2005.07702v1.pdf>
- [2] Blog Post <https://tobiassunderdiek.github.io/cartoon-gan/>
- [3] Cartoon Dataset <https://www.kaggle.com/datasets/alamson/safebooru>
- [4] Real Image Dataset <https://www.kaggle.com/datasets/hsankesara/flickr-image-dataset>